

Activity 10 - Parse CSV records with Result and custom errors

AI Vibe Coding with Rust / TGS-2023039924 / version 3.0 / K4/A4

Specification

`parse_record(&str) -> Result<Record, ParseError>` with `ParseError::`{`FieldCount(n)`, `EmptySku`, `BadNumber{field, value}`} and a Display message for each. `parse_all(&str)` skips blank lines and the header, returning valid records and (1-based line number, error) pairs.

Plan before code

Act as a senior Rust reviewer. Specification: `parse_record(&str) -> Result<Record, ParseError>` with `ParseError::`{`FieldCount(n)`, `EmptySku`, `BadNumber{field, value}`} and a Display message for each. `parse_all(&str)` skips blank lines and the header, returning valid records and (1-based line number, error) pairs. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Parse CSV records with Result and custom errors" in `starter/src` so that the supplied tests pass. Return `Result` everywhere; no `unwrap()`, `expect()` or `panic!` in library code. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no `unsafe`; no `unwrap()` on input; no new crates; every boundary tested; cargo test, clippy -D warnings and `fmt --check` pass; you can explain every line.